1. Problem Title & Source

Title: **Repeated Digit Sum**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

An intelligence agency receives encrypted reports represented as numbers.

Each report contains two values:

- **N** → a number
- **R** → number of repetitions

The decoding process works as follows:

1. Take the number **N**.
2. Compute the **sum of all digits of N**.
3. Replace the number with this sum.
4. Repeat this process **R times**.
5. Continue until the final result becomes a **single digit**.

Your task is to **determine the final single-digit value after performing the digit-sum operation R times**.

3. Examples

Example 1

Input

99

3

Output

9

Explanation

Step 1

Digit sum of 99

9 + 9 = 18

Step 2

Digit sum of 18

1 + 8 = 9

Step 3

Digit sum of 9

= 9

Final result

9

Example 2

Input

1234

2

Output

1

Explanation

Step 1

1 + 2 + 3 + 4 = 10

Step 2

1 + 0 = 1

Result

1

4. Constraints

$0 < N \leq 1000$

$0 \leq R \leq 50$

Observations

- Maximum digits ≈ **4**
- Digit sum reduces number quickly

5. Core Concept (Pattern)

Digit Manipulation

Important operations

modulo (%)

division (/)

6. Thought Process

Step 1 — Compute Digit Sum

Example

N = 99

Digit sum

9 + 9 = 18

Step 2 — Repeat R Times

If

R = 3

We repeat the process **three times**.

Step 3 — Reduce to Single Digit

Digit sum keeps reducing the number.

Example

18 → 9

Eventually it becomes **a single digit**.

7. Visualization

Example

N = 99

R = 3

Process

99 → 18 → 9 → 9

Final result

9

8. Pseudocode

```
read N
read R

repeat R times

   sum = 0

   while N > 0
      sum += N % 10
      N = N // 10

   N = sum

print N
```

9. Code Implementation

Python

```
n = int(input())
r = int(input())
```

```
for _ in range(r):

    s = 0

    while n > 0:
        s += n % 10
        n //= 10

    n = s

print(n)
```

Java

```
import java.util.*;

class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int r = sc.nextInt();

        for(int i = 0; i < r; i++){

            int sum = 0;

            while(n > 0){

                sum += n % 10;
                n /= 10;
            }

            n = sum;
        }
```

```
        System.out.println(n);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(R × digits)

Since

digits ≤ 4

the runtime is very small.

Space Complexity

O(1)

11. Common Mistakes

1. Not resetting sum inside loop
2. Forgetting to update n = sum
3. Using string conversion unnecessarily
4. Missing loop for R iterations

12. Detailed Dry Run

Input

N = 1234

R = 2

Step 1

1 + 2 + 3 + 4 = 10

Step 2

1 + 0 = 1

Result

1

13. Real-Life Applications

Digit sum operations appear in:

- Digital root calculations
- Checksum algorithms
- Error detection codes

- Number theory problems

14. Common Traps

1. Infinite loop if n not updated
2. Forgetting R iterations
3. Misinterpreting the problem as **repeat until single digit only**

15. Related Problems

Digit manipulation problems:

LC 258 Add Digits

LC 1281 Subtract Product and Sum of Digits

LC 66 Plus One

16. Alternate Approaches

Approach	Complexity	Notes
Repeated digit sum	O(R × digits)	simple
Digital root formula	O(1)	optimized math

Digital root formula

n % 9

17. Why This Solution Works

Each iteration reduces the number by **collapsing multiple digits into their sum**.

This guarantees that the number quickly reduces to a **single-digit value**.

18. Variations / Follow-Up Questions

Possible interview follow-ups:

1. Find **digital root of a number**
2. Repeat digit sum until **single digit**
3. Multiply digits repeatedly
4. Sum digits of **very large number (string input)**